## Experiment -17:

## Fast Fourier Transform Using DIT and DIF Algorithms

**OBJECTIVES**

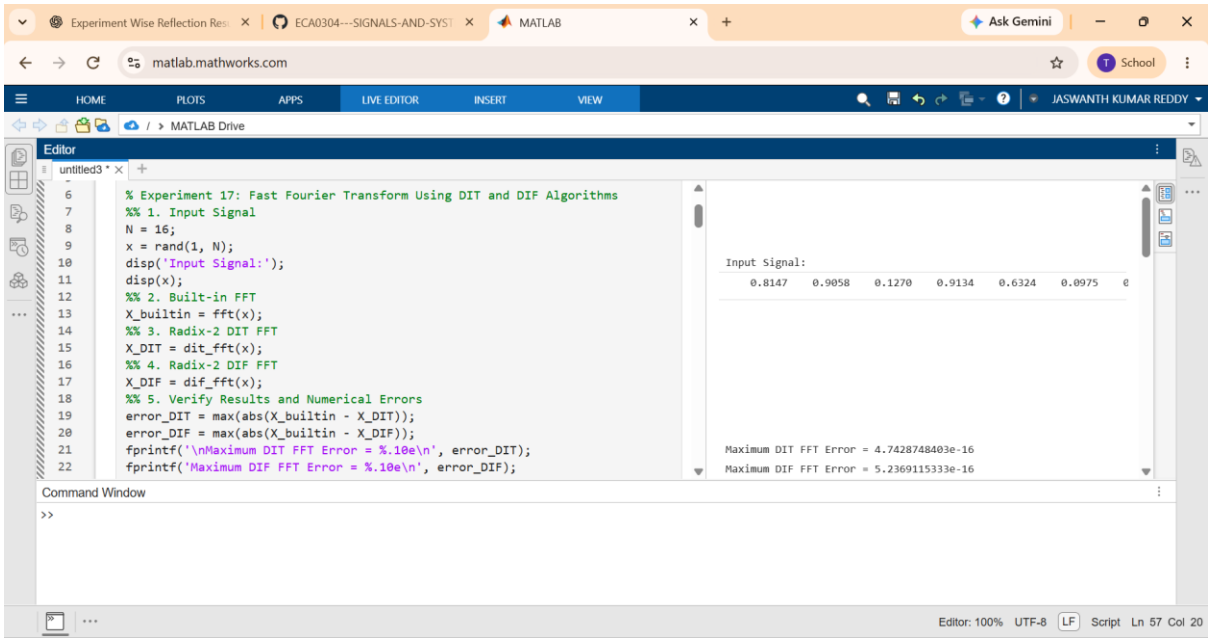
• Implement the Radix-2 Decimation-In-Time (DIT) FFT algorithm from first principles.

• Implement the Radix-2 Decimation-In-Frequency (DIF) FFT algorithm from first principles.

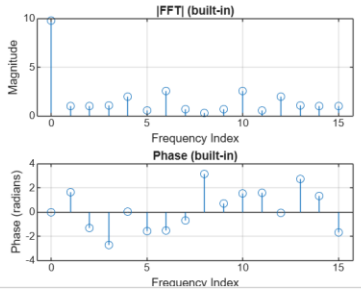
• Verify both custom implementations against the built-in fft() and quantify numerical error.

• Compare direct DFT and FFT computational effort and execution time as N increases.

**Output Graph :**



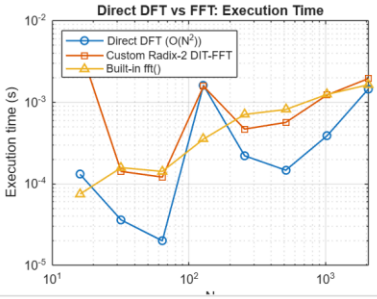
```
fprintf('Maximum DIF FFT Error = %.10e\n', error_DIF);
%% 6. Plot Magnitude and Phase
figure;
subplot(2,1,1);
stem(0:N-1, abs(X_builtin), 'o');
title('|FFT| (built-in)');
xlabel('Frequency Index');
ylabel('Magnitude');
grid on;
subplot(2,1,2);
stem(0:N-1, angle(X_builtin), 'o');
title('Phase (built-in)');
xlabel('Frequency Index');
ylabel('Phase (radians)');
grid on;
%% 7. Execution Time Comparison
Ns = [16 32 64 128 256 512 1024 2048];
t_DFT = zeros(size(Ns));
```

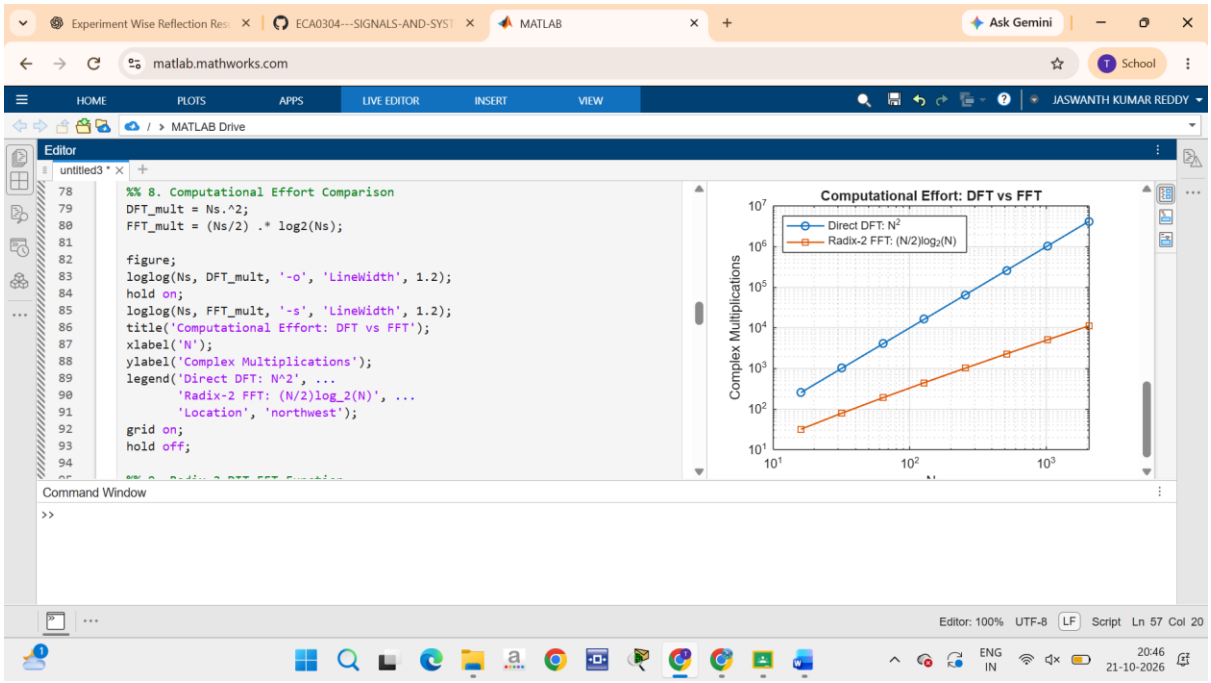



```
    t_builtin(k) = toc;
end

figure;
loglog(Ns, t_DFT, '-o', 'LineWidth', 1.2);
hold on;
loglog(Ns, t_FFT, '-s', 'LineWidth', 1.2);
loglog(Ns, t_builtin, '-^', 'LineWidth', 1.2);
title('Direct DFT vs FFT: Execution Time');
xlabel('N');
ylabel('Execution time (s)');
legend('Direct DFT (O(N^2))', ...
       'Custom Radix-2 DIT-FFT', ...
       'Built-in fft()', 'Location', 'northwest');
grid on;
hold off;

%% 8. Computational Effort Comparison
```






**Result:** Decimation-in-Time (DIT) and Decimation-in-Frequency (DIF) FFT algorithms were implemented to calculate the discrete Fourier transform efficiently.